Status: PREPARED DRAFT

Session 12 — Study Guide: Observable, Describable, Controlled

Where the session ends

Every real write route on both Projects and Workspaces is protected by a real, non-auth Guard, with EVERY mutation that can succeed — create, update, AND delete — logging a real, meaningful line, and the entire API (including both resources' own filter query parameters) fully documented through a live Swagger/OpenAPI document. One route, `GET /projects/stats`, is also guarded by homework's own end — a deliberate, reasoned exception to "only writes need protection." By the end of class, both resources' own create/read/update/delete/filter behavior from Session 11B is unchanged and still fully MongoDB-backed — this session adds observability and access control on top of it, not instead of it.

The real starting point

Session 11B's real classroom delivery finished Projects' own CRUD migration but never reached Workspaces or DELETE on either resource — both were assigned as take-home work instead. Today opens by genuinely solving that assignment together, live, before any of this session's own new material begins.

Logger — observe what happens

TS

```
private readonly logger = new Logger(ProjectsService.name);
```

A log line only earns its place at a genuinely meaningful point — a real, successful mutation, not every method call. Watch the server terminal, not just the HTTP response, the next time a create or delete succeeds.

Swagger/OpenAPI — describe and test what the API exposes

TS

```
const swaggerConfig = new DocumentBuilder()
  .setTitle('B4F Board API')
  .setDescription('Projects and Workspaces, backed by MongoDB.')
  .setVersion('1.0')
  .build();
const swaggerDocument = SwaggerModule.createDocument(app, swaggerConfig);
SwaggerModule.setup('docs', app, swaggerDocument);
```

`SwaggerModule.createDocument` reads every `@ApiTags` / `@ApiOperation` /DTO decorator already in the app to build a real, live OpenAPI document — `/docs` is for a human to read; `/docs-json` is the same document a tool could consume automatically. Swagger describes and lets you test the API; it does not

implement it — every decorator is purely descriptive metadata, never new logic.

A first Guard — control whether a request may reach a handler

TS

```ts
export class InternalToolGuard implements CanActivate {
  constructor(private readonly config: ConfigService) {}

  canActivate(context: ExecutionContext): boolean {
    const request = context.switchToHttp().getRequest<Request>();
    const providedKey = request.headers['x-internal-tool-key'];
    const expectedKey = this.config.get<string>('INTERNAL_TOOL_KEY');

    if (!providedKey || providedKey !== expectedKey) {
      throw new ForbiddenException(
        "This operation is restricted to B4F Board's own internal tooling.",
      );
    }

    return true;
  }
}
```

This is access control, deliberately **not** authentication:

```
temporary internal access control
        ≠
authentication
```

It answers exactly one question — does this request carry the internal tool's own shared key — never who is making the request. There is no user, no identity, no session anywhere in this class. That is a separate, later topic (real JWT authentication).

Proving it

BASH

```bash
curl -i -X POST localhost:3000/projects -H 'Content-Type: application/json' -d
'{"name":"Guard Test","key":"GRD"}'
curl -i -X POST localhost:3000/projects -H 'Content-Type: application/json' -H 'x-internal-
tool-key: <the real local value>' -d '{"name":"Guard Test","key":"GRD"}'
```

The first request returns `403` and produces **no log line at all** from `ProjectsService` — the handler, and everything inside it, never ran. The second succeeds and logs a real, meaningful line. Log silence on rejection is the actual proof a Guard works, not just the status code alone. The same two-sided proof

applies to `update` too, once it's guarded — a rejected `PATCH` produces no `Updated project ...` line; an accepted one does. (A real local value for `INTERNAL_TOOL_KEY` must already exist in your own `.env` — never `.env.example` — before any of this can succeed.)

Common beginner mistakes

- **Logging in every method "for completeness."** A log earns its place by marking something genuinely worth observing; logging everything is noise, not observability.
- **Confusing `@ApiProperty` with `@IsString()` / `@IsNotEmpty()`.** One documents a field for Swagger; the other actually validates it. Neither replaces the other.
- **Treating a `403` as proof the route is broken.** It's proof the Guard is working — verify by then supplying the real header.
- **Using `@nestjs/mapped-types`'s own `PartialType` after adding Swagger decorators.** Only `@nestjs/swagger`'s own `PartialType` carries that metadata into an Update DTO.
- **Assuming the Guard identifies a person.** It does not — it checks whether the right shared key was presented, nothing about identity.
- **Logging create and delete but forgetting update.** A successful mutation that logs nothing is just as unobservable as one that's silently rejected — "every mutation that can succeed logs" means all three operations, not two of three.

Not yet

These exist in NestJS, but are deliberately **not** covered this session:

- Custom decorators
- Interceptors
- Real authentication (JWT, users, sessions, password hashing, roles/permissions)
- Guards applied to real user identity/roles
- Database-level uniqueness constraints and indexes
- MongoDB's own `_id` as a public identifier
- Aggregation pipelines, deeper embedding/references
- PostgreSQL, an ORM, Next.js

Glossary

- **`Logger`** — Nest's own observability class; `new Logger(ClassName.name)` scopes every log line with its own origin.
- **`DocumentBuilder` / `SwaggerModule`** — builds and mounts a real OpenAPI document and browsable UI from decorator metadata already in the app.
- **`@ApiTags` / `@ApiOperation` / `@ApiProperty` / `@ApiHeader` / `@ApiResponse`** — purely descriptive Swagger decorators.
- **`CanActivate`** — the contract a Guard implements; `canActivate()` runs before any handler.

- `@UseGuards(...)` — applies a Guard to a route; the Guard class must be registered as a provider in its own module.
- **Access control vs. authentication** — access control answers "is this request allowed right now"; authentication answers "who is making this request."

Review questions

1. Why does Logger belong only at genuinely meaningful points, not in every method?
2. What does Swagger actually do to the API's own behavior?
3. Why is `InternalToolGuard` access control, not authentication?
4. What two things does a rejected request have to prove, not just one?
5. Why must `UpdateProjectDto`'s own `PartialType` come from `@nestjs/swagger`, specifically?

Answers

1. A log is meant to mark a genuinely meaningful, real event (a successful mutation); logging every call buries the meaningful lines in noise and defeats the whole purpose of observability.
2. Nothing — Swagger decorators are purely descriptive metadata read to build a document; they never change what a route validates or does.
3. It checks only whether the request carries the correct shared key — it never asks who the caller is; authentication (JWT, still to come) answers that separate question.
4. A `403` status code, AND no log line at all from the service the request was rejected before reaching.
5. `@nestjs/mapped-types`'s own `PartialType` only carries `class-validator` metadata into the subclass; `@nestjs/swagger`'s own version additionally carries the `@ApiProperty` metadata, so the Update DTO's own Swagger document isn't empty.

What you should be able to explain now

- Why Logger, Swagger, and a Guard form one coherent arc, not three unrelated features.
- Why a Guard's rejection must be proven two ways: status code AND log silence.
- Why access control and authentication are genuinely different questions, not two names for the same thing.
- Why not every real access-control decision is about write operations.